Dokumentacja

Gra „Podziemny Labirynt” – Mark Viediernikov

Przedmiot: Języki zorientowane-obiektowo

---

1. Wprowadzenie

Projekt to prosta gra typu „dungeon crawler” (mała gra RPG z pokojami i korytarzami) dla jednego gracza, działająca w terminalu. Została napisana w Pythonie i korzysta wyłącznie z biblioteki standardowej (`abc`, `enum`, `os`) – nie wymaga instalowania żadnych dodatkowych pakietów. Mapa jest rysowana znakami tekstowymi, a gracz steruje bohaterem, walczy z wrogami, zbiera przedmioty i przechodzi przez dwa poziomy.

Kod jest podzielony na czternaście modułów: `main.py`, `engine.py`, `input_handler.py`, `commands.py`, `game_map.py`, `renderer.py`, `actor.py`, `hero.py`, `enemy.py`, `item.py`, `weapon.py`, `potion.py`, `key.py` oraz `non_copyable.py`. Każda klasa znajduje się w osobnym pliku. Dodatkowo wszystkie klasy mają zablokowane kopiowanie obiektów.

2. Instrukcja uruchomienia

2.1 Wymagania i uruchomienie

Wymagania: Python 3.8 lub nowszy. Biblioteki zewnętrzne nie są potrzebne.

Kroki:

1. Skopiować wszystkie pliki `.py` projektu do jednego folderu.
2. Otworzyć terminal (cmd, PowerShell lub terminal systemowy) w tym folderze.
3. Uruchomić grę:

```
python main.py
```

W systemach Linux i macOS może być potrzebne polecenie `python3 main.py`.

2.2 Sterowanie

- **W / A / S / D** + Enter – ruch bohatera (góra, lewo, dół, prawo).
- **Q** + Enter – wyjście z gry.
- Ruch w stronę wroga oznacza atak.

2.3 Zasady gry

- **Poziom 1:** pokonaj wszystkich wrogów, a następnie dojdź do przejścia `|` w prawym dolnym rogu mapy. Jeżeli na mapie są jeszcze żywi wrogowie, przejście nie działa – gra wyświetla komunikat, a bohater zostaje cofnięty o jedno pole.
- **Poziom 2:** zbierz klucz `K` w lewym górnym pokoju, wejdź w drzwi `?` (bez klucza drzwi się nie otworzą) i pokonaj wszystkich wrogów. Wtedy gra kończy się zwycięstwem.
- **Porażka:** punkty życia (HP) bohatera spadają do zera.
- Bohater zaczyna z 10 HP i zadaje 2 obrażenia (z mieczem 6). Wróg ma 5 HP i zadaje 1 obrażenie. Wrogowie znajdujący się w odległości do 6 pól ruszają w stronę bohatera i atakują, gdy staną obok niego.
- Stan bohatera (HP i wyposażenie) zostaje zachowany po przejściu na drugi poziom.

2.4 Legenda symboli

Symbol	Znaczenie
`@`	bohater (gracz)
`E`	wróg
`/`	miecz – zwiększa siłę ataku o 4
`!`	mikstura – przywraca do 15 punktów życia (nie więcej niż maksimum)
`K`	klucz do drzwi na drugim poziomie
`?`	drzwi – otwierane kluczem
`|`	przejście na drugi poziom (wymaga pokonania wszystkich wrogów)
`#`	ściana
`.`	podłoga

3. Ogólna architektura

Program jest zbudowany wokół klasy silnika `Engine` oraz trzech pomocniczych klas, którym silnik deleguje zadania, a także dwóch hierarchii dziedziczenia (postacie i przedmioty):

- `Engine` – silnik gry: główna pętla, tury bohatera i wrogów, przechodzenie między poziomami. Sam nie czyta klawiszy ani nie rysuje – koordynuje pozostałe klasy.
- `InputHandler` (razem z `Command`) – odczyt wejścia od gracza i zamiana go na polecenie.
- `Map` – mapa: układ ścian i drzwi oraz przedmioty leżące na planszy; wie, którędy można przejść.
- `Renderer` – rysowanie mapy, postaci i paska stanu w terminalu.
- `Actor` → `Hero`, `Enemy` – postacie w grze.
- `Item` → `Weapon`, `Potion`, `Key` – przedmioty do zebrania.
- `NonCopyable` – klasa bazowa blokująca kopiowanie obiektów.

Pełna hierarchia dziedziczenia:

```
NonCopyable
 ├── Actor  (abstrakcyjna)
 │     ├── Hero
 │     └── Enemy
 ├── Item   (abstrakcyjna)
 │     ├── Weapon
 │     ├── Potion
 │     └── Key
 ├── Map
 ├── InputHandler
 ├── Renderer
 └── Engine
```

Kluczowa idea: każda klasa ma jedno, jasno określone zadanie. Dzięki temu zmiana wyglądu ekranu dotyczy tylko `Renderer`, zmiana klawiszy tylko pliku `commands.py`, a zmiana układu poziomu tylko list `LEVEL_ONE` i `LEVEL_TWO` w `engine.py`.

4. Zasady OOP zastosowane w projekcie

4.1 Abstrakcja

Klasy `Actor` i `Item` są klasami abstrakcyjnymi. Nie da się utworzyć ich obiektu – próba kończy się błędem `TypeError`. Zawierają metody oznaczone dekoratorem `@abstractmethod`, które określają, **co** ma zrobić klasa pochodna, ale nie **jak**.

- `Actor.attack_power()` – każda postać sama określa siłę swojego ataku.
- `Item.on_pickup(hero)` – każdy przedmiot sam określa, co się dzieje po jego podniesieniu.

```
class Actor(NonCopyable, ABC):
    ...
    @abstractmethod
    def attack_power(self): ...
```

4.2 Dziedziczenie

Klasy pochodne dziedziczą pola i metody po klasach bazowych i dodają własne. Relacja „jest”:

- `Hero` i `Enemy` są klasami `Actor` – dziedziczą imię, symbol, pozycję, punkty życia oraz metody `move_to()`, `take_damage()`, `heal()` i `attack()`.
- `Weapon`, `Potion` i `Key` są klasami `Item` – dziedziczą nazwę, symbol i pozycję.
- Wszystkie klasy projektu dziedziczą po `NonCopyable`.

Konstruktor klasy bazowej jest wywoływany przez `super().__init__(...)`.

```
class Hero(Actor):
    def __init__(self, x, y):
        super().__init__("Bohater", "@", x, y, 10)
        self.__weapon = None
        self.__key = None
```

4.3 Polimorfizm

Ten sam kod wywołuje różne zachowania w zależności od rzeczywistego typu obiektu:

- `Actor.attack()` wywołuje `attack_power()`. Dla `Hero` wynik zależy od posiadanej broni (2 lub 2 + bonus miecza), a dla `Enemy` jest stały.
- `Engine` wywołuje `item.on_pickup(hero)`, nie wiedząc, jaki to przedmiot: miecz zostanie założony (`equip`), mikstura wyleczy bohatera (`heal`), a klucz trafi do jego ekwipunku (`getkey`).

```
def attack(self, target):
    target.take_damage(self.attack_power())

item.on_pickup(self.__hero)   # Weapon, Potion lub Key
```

4.4 Hermetyzacja

Stan obiektów jest chroniony przed bezpośrednią modyfikacją z zewnątrz. Zmienia się go wyłącznie przez metody publiczne, np. `take_damage()`, `heal()`, `move_to()`, `equip()`, `open_door()`. Odczyt pól umożliwiają właściwości tylko do odczytu (`@property`), np. `hero.hp`, `actor.alive`, `item.symbol`.

Poziom dostępu	Zapis w Pythonie	Przykłady w projekcie
publiczny	`nazwa`	`Engine.run()`, `Actor.attack()`, `Map.is_wall()`
chroniony (protected)	`_nazwa`	`Actor._hp`, `Actor._x`, `Item._symbol`
prywatny (private)	`__nazwa`	`Hero.__weapon`, `Enemy.__power`, `Map.__rows`, `Engine.__map`
tylko do odczytu	`@property`	`Actor.hp`, `Actor.x`, `Hero.key`, `Weapon.bonus`

Pola potrzebne klasom pochodnym (np. `_hp`, `_x`) są chronione, natomiast pola używane tylko wewnątrz jednej klasy (np. `__weapon`, `__rows`, `__items`) są prywatne. Metody pomocnicze silnika (`__load`, `__hero_turn`, `__enemies_turn`) również są prywatne. Publiczne jest tylko to, co musi być dostępne z zewnątrz.

W Pythonie modyfikatory dostępu są konwencją. Pojedyncze podkreślenie oznacza „chronione”, a podwójne uruchamia mechanizm zmiany nazwy (name mangling): pole `__weapon` w klasie `Hero` jest w rzeczywistości przechowywane jako `_Hero__weapon`. Dzięki temu nie można go przypadkowo nadpisać.

4.5 Blokada kopiowania

Obiekty w grze są unikalne – kopia bohatera oznaczałaby dwóch bohaterów, a kopia silnika dwie równoległe gry. Dlatego wszystkie klasy dziedziczą po `NonCopyable`, która nadpisuje metody `__copy__` i `__deepcopy__` (używane przez `copy.copy()` i `copy.deepcopy()`) i zgłasza wyjątek `TypeError`. Zabezpieczenie jest zapisane w jednym miejscu i działa we wszystkich klasach dzięki dziedziczeniu.

```
class NonCopyable:
    def __copy__(self):
        raise TypeError(f"{type(self).__name__} nie mozna skopiowac")

    def __deepcopy__(self, memo):
        raise TypeError(f"{type(self).__name__} nie mozna skopiowac")
```

Zwykłe przypisanie (`b = a`) nie tworzy kopii – obie zmienne wskazują ten sam obiekt – dlatego nie wymaga blokowania.

4.6 Kompozycja i agregacja

Złożone obiekty są budowane z mniejszych, zamiast być tworzone przez dziedziczenie:

- `Engine` zawiera `InputHandler`, `Renderer`, `Map`, jednego `Hero` oraz listę obiektów `Enemy` (kompozycja) – tworzy je i zarządza ich życiem.
- `Map` przechowuje listę obiektów `Item` leżących na planszy (agregacja).
- `Hero` przechowuje zdobytą broń (`Weapon`) i klucz (`Key`) (agregacja).

4.7 Podział odpowiedzialności i konfigurowalne sterowanie

Klawisze nie są zapisane w logice klasy `InputHandler`. Polecenia gracza to typ wyliczeniowy `Command`, a przypisanie klawiszy do poleceń jest słownikiem `DEFAULT_KEYS` w pliku `commands.py`. `InputHandler` przyjmuje ten słownik w konstruktorze, więc zmiana

klawisza (np. z `w` na `i`) nie wymaga ingerencji w kod klasy, a `Engine` w ogóle nie wie, jakich liter używa gracz – otrzymuje tylko `Command.UP`, `Command.LEFT` itd.

```
def __init__(self, keys=None):
    if keys is None:
        keys = DEFAULT_KEYS
    self.__keys = dict(keys)
```

5. Opis poszczególnych modułów

5.1 main.py

Punkt wejścia programu. Tworzy obiekt `Engine` i wywołuje jego metodę `run()`, która uruchamia całą grę.

5.2 engine.py

Klasa `Engine` to silnik gry. Zawiera dwie stałe z układami poziomów (`LEVEL_ONE`, `LEVEL_TWO`) oraz słownik `MOVES`, który zamienia polecenie ruchu na przesunięcie `(dx, dy)`. W układzie poziomu znaki `#` i `.` opisują teren, a znaki `@`, `E`, `S`, `P`, `K` są znacznikami, z których podczas ładowania powstają obiekty (odpowiednio: bohater, wróg, miecz, mikstura, klucz). Znaki `?` (drzwi) i `|` (przejście) pozostają na mapie jako elementy terenu.

Najważniejsze metody:

- `run()` – główna pętla gry. Dopóki bohater żyje: rysuje ekran, odczytuje polecenie, wykonuje turę bohatera, a następnie turę wrogów. Po każdej turze sprawdza warunki przejścia na poziom 2 (bohater stoi na przejściu i nie ma żywych wrogów) oraz zwycięstwa (poziom 2 bez żywych wrogów). Na końcu wyświetla komunikat o wygranej lub przegranej.
- `__load(layout)` – zamienia tekstowy układ poziomu na obiekty. Przy pierwszym wczytaniu tworzy bohatera, a przy kolejnym tylko przesuwa istniejącego na nowe pole startowe, dzięki czemu jego HP i wyposażenie zostają zachowane. Na końcu usuwa znaczniki z terenu i tworzy obiekt `Map`.
- `__hero_turn(dx, dy)` – jeśli w docelowym polu stoi wróg, bohater go atakuje. Jeśli pole nie jest ścianą: przy drzwiach sprawdza posiadanie klucza (z kluczem otwiera drzwi i wchodzi, bez klucza wyświetla komunikat); w pozostałych przypadkach przesuwa bohatera i podnosi ewentualny przedmiot przez `item.on_pickup(hero)`.
- `__enemies_turn()` – każdy żywy wróg pyta siebie o kolejny krok (`next_step`). Jeśli krok prowadzi na pole bohatera, wróg atakuje. Jeśli pole jest wolne (nie jest ścianą, drzwiami ani polem zajętym przez innego wroga), wróg się przesuwa.
- `__enemy_at(x, y)`, `__enemies_left()`, `__draw()` – metody pomocnicze.

5.3 input_handler.py i commands.py

`commands.py` zawiera typ wyliczeniowy `Command` (UP, DOWN, LEFT, RIGHT, QUIT, NONE) oraz słownik `DEFAULT_KEYS` przypisujący klawisze do poleceń (`w`, `a`, `s`, `d`, `q`). `InputHandler.read()` czeka na wpisanie tekstu, usuwa zbędne spacje, zamienia litery na małe, bierze pierwszy znak i zwraca odpowiadające mu polecenie. Dla pustego lub nieznanego wejścia zwraca `Command.NONE`, a przy zakończeniu strumienia wejścia (`EOFError`) – `Command.QUIT`.

5.4 game_map.py

Klasa `Map` przechowuje prywatną listę wierszy z terenem (`__rows`) oraz listę przedmiotów leżących na planszy (`__items`). Udostępnia metody: `is_wall(x, y)` – czy pole jest ścianą, `is_door(x, y)` – czy pole jest drzwiami, `open_door(x, y)` – zamienia drzwi na podłogę, `item_at(x, y)` – zwraca przedmiot z danego pola lub `None`, `remove_item(item)` – usuwa przedmiot po podniesieniu oraz `row(y)`, `height`, `items` – potrzebne do rysowania. Współrzędna `x` oznacza kolumnę, a `y` wiersz (w kodzie `rows[y][x]`).

5.5 renderer.py

Klasa `Renderer` odpowiada wyłącznie za rysowanie i niczego nie zmienia w stanie gry. Metoda `draw()` buduje płótno jako listę list znaków, na które w kolejności nakłada: teren, przedmioty, wrogów i na końcu bohatera (dzięki temu bohater jest zawsze widoczny). Następnie czyści ekran (`cls` w Windows, `clear` w pozostałych systemach), wypisuje mapę, pasek stanu (HP, broń, liczba wrogów), legendę, sterowanie i bieżący komunikat.

5.6 actor.py, hero.py, enemy.py

`Actor` to abstrakcyjna klasa bazowa postaci. Przechowuje chronione pola `_name`, `_symbol`, `_x`, `_y`, `_hp`, `_max_hp` i udostępnia je przez właściwości. Zawiera metody `move_to()`, `take_damage()` (HP nie spada poniżej zera), `heal()` (HP nie przekracza maksimum) oraz `attack(target)`, która korzysta z abstrakcyjnej metody `attack_power()`. Właściwość `alive` sprawdza, czy `hp > 0`.

`Hero` (10 HP, symbol `@`) dodaje prywatne pola `__weapon` i `__key`, metody `equip()` i `getkey()` oraz właściwości `weapon_name` i `key`. Siła ataku wynosi 2 lub 2 + bonus założonej broni.

`Enemy` (5 HP, symbol `E`, siła ataku 1) dodaje metodę `next_step(target)`. Zwraca ona pole, do którego wróg chce pójść: jeśli bohater jest dalej niż 6 pól (suma odległości w poziomie i pionie), wróg stoi w miejscu, w przeciwnym razie idzie w stronę bohatera po osi, na której dzieli ich większa odległość. Metoda tylko wskazuje pole – o możliwości wejścia na nie (ściany, inni wrogowie, drzwi) decyduje `Engine`.

5.7 item.py, weapon.py, potion.py, key.py

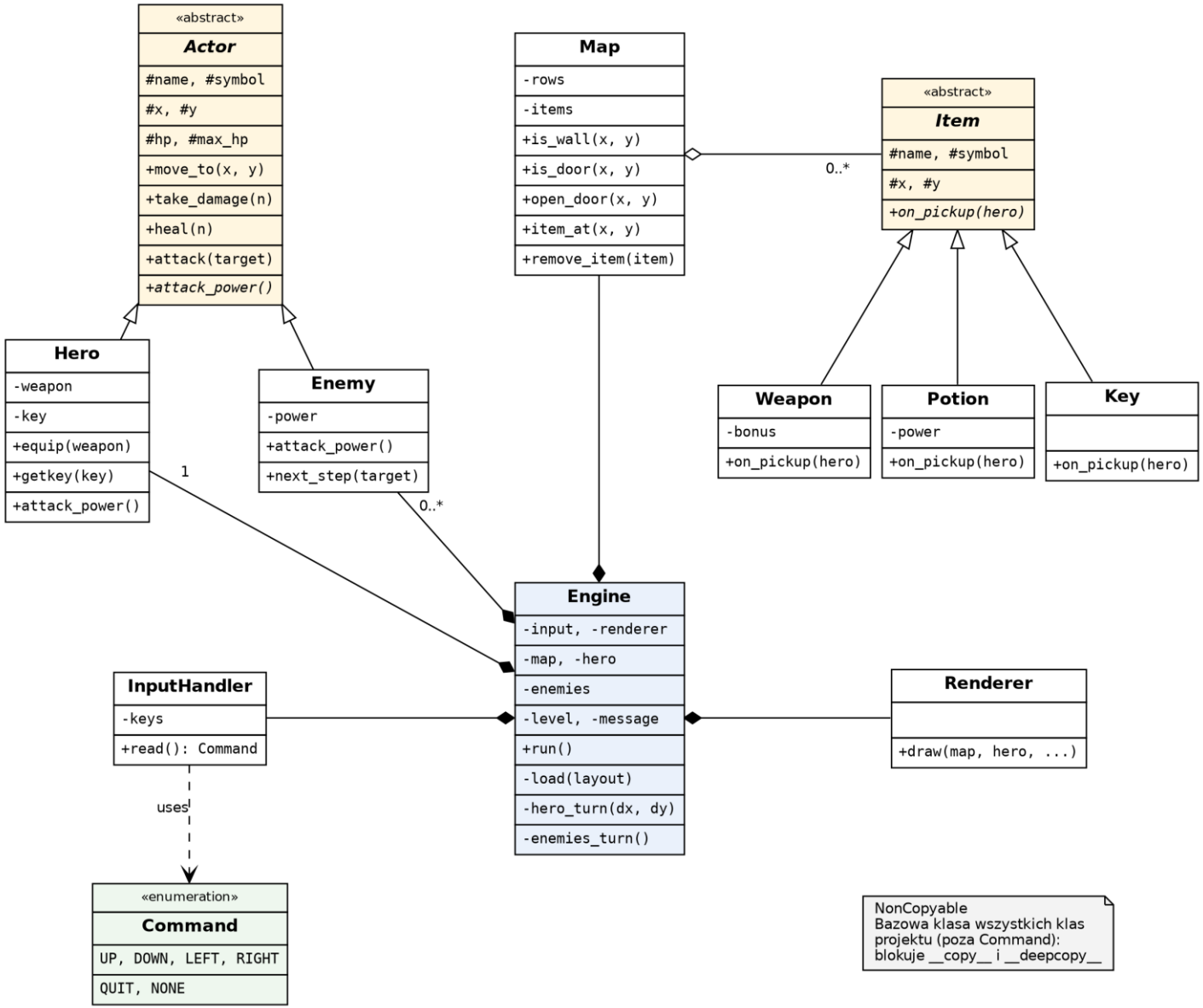
`Item` to abstrakcyjna klasa bazowa przedmiotów z chronionymi polami `_name`, `_symbol`, `_x`, `_y` i abstrakcyjną metodą `on_pickup(hero)`. Klasy pochodne:

- `Weapon` (symbol `/`) – ma prywatny bonus do ataku `__bonus`; po podniesieniu wywołuje `hero.equip(self)`. W grze występuje miecz z bonusem +4.
- `Potion` (symbol `!`) – ma prywatną moc leczenia `__power`; po podniesieniu wywołuje `hero.heal(...)`. Mikstura leczy do 15 HP.
- `Key` (symbol `K`) – po podniesieniu wywołuje `hero.getkey(self)`, co pozwala otworzyć drzwi na drugim poziomie.

5.8 non_copyable.py

Klasa bazowa blokująca kopiowanie (opisana w punkcie 4.5). Wszystkie pozostałe klasy projektu dziedziczą po niej pośrednio lub bezpośrednio.

6. Diagram klas (UML)



6.1 Objaśnienie diagramu i legenda

Diagram przedstawia wszystkie klasy projektu wraz z ich najważniejszymi polami i metodami. Kolorem żółtym zaznaczono klasy abstrakcyjne, a niebieskim silnik gry. Znak `#` oznacza pole chronione, `-` prywatne, `+` publiczne, a kursywa – element abstrakcyjny.

Legenda:

- Strzałka z pustym trójkątnym grotem – dziedziczenie (np. `Hero` dziedziczy po `Actor`).
- Pełny romb – kompozycja: `Engine` zawiera `InputHandler`, `Renderer`, `Map`, `Hero` (1) i `Enemy` (0..*).
- Pusty romb – agregacja: `Map` przechowuje przedmioty `Item` (0..*).
- Linia przerywana ze strzałką – zależność (`InputHandler` używa typu `Command`).
- Notatka `NonCopyable` – klasa bazowa wszystkich klas z diagramu poza `Command` (strzałek dziedziczenia nie narysowano, aby nie zaciemniać schematu).
- Dla czytelności pominięto właściwości `@property` oraz agregację `Hero` → `Weapon` i `Key`.

7. Możliwe rozszerzenia

7.1 Możliwe rozszerzenia

- **Nowy przedmiot** (np. tarcza): utworzyć klasę dziedziczącą po `Item`, zaimplementować `on_pickup()` i dodać nowy znacznik w metodzie `__load`.
- **Nowy poziom:** dodać kolejną listę znaków (np. `LEVEL_THREE`) i obsłużyć przejście w metodzie `run()`.
- **Nowy typ wroga:** utworzyć klasę dziedziczącą po `Actor` z własną implementacją `attack_power()` i własnym sposobem poruszania się.
- **Inne sterowanie** (np. odczyt pojedynczych klawiszy bez Entera lub strzałki): zmiana ogranicza się do `InputHandler` i `commands.py`, ponieważ `Engine` operuje wyłącznie na obiektach `Command`.
- **Wstrzykiwanie zależności:** obecnie `Engine` sam tworzy `InputHandler` i `Renderer`. Przekazanie ich przez konstruktor pozwoliłoby wymieniać je bez modyfikacji silnika.